EMT 3104: Mechatronic Systems Programming I - Notes

Lesson 1:

DEFINITION OF MECHATRONIC SYSTEMS

Mechatronics combines various disciplines in engineering, including mechanics, computing/programming, control, and electronics. Mechatronic systems are thus mechanical and electrical systems that are controlled through programming implemented using sensors, actuators and other embedded electronic devices.

Programming is the design and implementation of computer understandable instructions in order to achieve efficient and low cost operation of systems. These computer instructions are written using programming languages which are specific to different embedded systems and controllers. Examples of embedded system controllers include Microprocessors (μP), Microcontrollers (μC), Programmable Logic Controllers (PLC), and Field Programmable Gate Arrays (FPGA).

WHAT IS EMBEDDED PROGRAMMING?

Embedded programming is the term for the computer programming that is used to program computer-controlled devices in day to day applications i.e. homes, cars, workplaces and communities. For complex embedded systems such as desktops, laptops or tablet computer there are dozens of microcontrollers quietly performing embedded tasks e.g. wireless communication, audio and visual signal processing, input device interfacing etc. all under the control of one main controller.

All these functionality needs to be achieved through writing programs and thus there is great need for embedded programming. Microcontrollers enable the control to countless devices and systems, enhancing those devices and systems to operate better, faster, more safely, more efficiently, more conveniently and in many cases allowing the very existence of devices and systems that could not be built otherwise. Spend some time looking around in the modern world and it is easy to recognize where μCs are working how ubiquitous this technology has become since their invention some 40+ years ago. On top of all that, μC programming is a fascinating and rewarding branch of the programming tree which is a necessary skill to master in becoming a complete Mechatronic Engineer.

WHAT IS AN EMBEDDED SYSTEM?

There's no perfect answer to that question, since every answer will have some exceptions. However, for our purposes let us declare that an embedded system is one that uses one or more microcomputers running custom dedicated programs and connected to specialized hardware, to perform a dedicated set of functions. This can be contrasted with a general-purpose computer such as the familiar desktop or notebook, which are not designed to run only one dedicated program with one specialized set of hardware. Due to the continued changes in current technology, this definition continues to be refined. Some examples of embedded systems are:

- Alarm / security system
- Automobile cruise control
- Heating / air conditioning thermostat
- Microwave oven
- Traffic light controller
- Vending machine
- Gas pump
- Game console (Play Station)
- Irrigation system controller
- Oscilloscope
- Mars Rover

In general these are examples sourced from the less complex end of the embedded systems spectrum. By the end of this unit you should have a good idea how most of these applications would be programmed, and in rough terms what kinds of input and output (I/O), timing, interrupt and communications hardware and functionality they would require. In order to facilitate human interaction, traditional input-output devices are generally used (keyboards, mice, displays) and at times need to be in cooperated into embedded systems.

WHAT IS DIFFERENT ABOUT EMBEDDED PROGRAMMING?

Embedded programs must work closely with the specialized components and custom circuitry that makes up the hardware. Unlike programming on top of a full-function operating system, where the hardware details are removed as much as possible from the programmer's notice and control, most embedded programming acts directly with and on the hardware. This includes not only the hardware of the CPU, but also the hardware which makes up all the peripherals (both on-chip and off-chip) of the system. Thus an embedded programmer must have good knowledge of hardware, at least as it pertains to writing software that correctly interfaces with that hardware. This knowledge will often extend to specifying key components of the hardware (microcontroller, memory devices, I/O devices, etc), and in smaller organizations will sometimes go as far as designing and laying out (as a printed circuit board) the hardware. An embedded programmer will also need to have a good understanding of typical debugging and testing equipment such as multimeter, oscilloscopes, logic analysers and the like.

Another difference from general purpose computers is that most (but not all) embedded systems are quite limited as compared to the former. The microcomputers used in embedded systems may have program memory sizes of a few thousand to a few hundred thousand bytes rather than the gigabytes in the desktop machine, and will typically have even less data (RAM) memory than program memory. Further, the CPU will often be smaller 8 and 16 bit devices as opposed to the 32 bit and larger devices found in a desktop (although small 32-bit microcontrollers are now under Ksh.200 in moderate quantities which is quite affordable). A smaller CPU word size means, among other things, that a program will require more instructions (and thus more clock cycles) than an equivalent program running on a CPU with a larger word size. And finally, the speed at which smaller microcontrollers run is much less than the speed at which a PC runs. Typical smaller microcontroller clock rates are between 1 and 200 MHz, not the GHz rates of PCs.

WHAT ARE THE DIFFERENCES BETWEEN MICROCOMPUTER, MICROPROCESSOR AND MICROCONTROLLER?

A microprocessor is usually understood to be a single-chip central processing unit (CPU), with the CPU being the "brains" of a computer - the part of the computer that executes program instructions. A microcomputer is any computer built around a microprocessor, along with program and data memory, and I/O devices and other peripherals as needed. A microcontroller (often shortened to µC) is a single chip device which has built onto the chip not only a microprocessor but also on the same chip, non-volatile program (ROM) and volatile data (RAM) memory, along with useful peripherals such as general-purpose I/O (GPIO), timers and serial communications channels. Thus it follows that all microcontrollers are microcomputers, but not all microcomputers use microcontrollers.

In smaller embedded systems it is most common to use microcontrollers rather than microprocessor-based designs since microcontrollers give the most compact design and the lowest hardware cost. Larger embedded systems, on the other hand, may use one or more microprocessors if a microcontroller of suitable speed and functionality cannot be found. This can extend to the use of industrial PCs and even more powerful hardware. It is also possible to include both microprocessors and microcontrollers in a complex embedded system. The only real rules are, use whatever device(s) fit the task, given the constraints on budget, availability, time, tools, etc.

It should also be pointed out that with most microcontrollers it is possible to add external memory and peripherals, should the on-board mix not take care of all the system needs. When it makes sense to add such external devices, as opposed to choosing a larger microcontroller with the needed resources on-board, is a choice that needs to be made on an individual design basis.

WHAT IS AN N-BIT CPU/MICROPROCESSOR/MICROCONTROLLER?

If the device can perform most of its data manipulation instructions on data words up to N bits in size, the device is an N-bit processor. By way of example, a device may have a full set of instructions that can operate on 8 bit data, along with a few instructions that operate on 16 bit data, that device should be considered an 8-bit design

By volume, 8-bit microcontrollers are the biggest segment of the embedded market. Many applications simply don't need any more power. 16-bit devices are more powerful, but they are squeezed between the 8-bit devices on the low end and the 32-bit devices on the high end. 32-bit devices are at the high end of the embedded spectrum for all but the most complex or high-performance designs, but their cost is gradually getting lower.

WHAT MICROCONTROLLER FAMILIES WILL WE CONSIDER?

To give a bit of an overview of the different flavours of microcontrollers available, this unit will be written around one 8-bit family (the Atmel AVR) i.e. Atmega32 and is generally coded in C language. Other popular microcontroller families do exist such as PIC and ARM microcontrollers but these are left for the student to consider. The main programming concepts remain the same it is just the syntax that changes.

WHAT ELSE IS REQUIRED FOR THIS CLASS?

A majority of this unit will use an electronic circuit simulator i.e. ***Proteus*** electronic development platform and a coding environment i.e. ***Atmel Studio*** 7 and above. Since it is best to practically verify the concepts learnt in class, it is strongly recommend that the students strives to obtain, either a microcontroller training/development board, or even just a bare µC chip, assorted components and a powered breadboard. (Making your own microcontroller board).

In addition you will need a C compiler that targets your device, and optionally an assembler for your device. You should have no trouble finding a free assembler for your chip, and you should also be able to find a free C compiler, even if it is a reduced-functionality version of a commercial compiler. You will also need a method of downloading your programs into your µC. The details of this download process will depend intimately on the particular µC and board it is mounted on. (Can generally be found on the internet)

As far as test equipment is concerned, devices such as digital multimeter are items that any mechatronic engineer should not do without. They can be purchased at local electronics shops at affordable prices. The other piece of equipment that is highly useful is an oscilloscope thus students need to familiarize themselves with how to use it effectively.

Regarding the microcontrollers used in this unit, here are the details of the hardware and we will be using:

AVR

•Hardware: Atmega32, in Proteus. •Tools: Atmel Studio 7 (free)

WHICH PROGRAMMING LANGUAGE?

This is a good time to talk a bit about the various programming languages that one can use to write embedded software. The early languages used in embedded programming are assembly language (ASM) and C. The main language used in this unit will be C. There are other languages that can be used in microcontroller programming that make programming easier. That being said, both C and assembly language are useful not only for learning about µC programming, but also for actually doing productive µC programming. They are also ubiquitous in that no matter what microcontroller you choose, it will almost certainly have available both an assembler (for processing assembly language source code) and a C compiler (for processing C source code). The same is definitely not the case for other languages. But I would encourage you to consider other languages if you are so inclined.

On the subject of assembly language, even if you don't plan on using assembly language in your embedded programming, I would strongly suggest that you become at least somewhat familiar with the concepts, and with the instruction set of your µC. The reason for this is that, even if you don't end up writing any assembly language you will find yourself at some point needing to examine the output of your compiler and/or your compiler-supplied start-up files written or output in assembly language.

Lesson 1 Review and Lesson 2 Preview

1st Report: A Review of FPGAs and PLCs
5 page (Excluding title page and reference page)
2 Weeks submission Deadline: Submission Date –
Include diagrams, pictures and flowcharts.
Give an example/sample

Lesson 2:

HOW DOES AN EMBEDDED PROGRAM RUN?

Before talking much more about embedded programming, this is a good place to give a brief overview of how an embedded program starts up and runs. Assuming that you have generated a program file and have loaded it into the µC program memory (all steps that we will talk about in more detail later), the good stuff happens when you either turn on the device or you push the RESET button. When the µC comes out of reset from either action it will always go to a particular memory location, as defined by the manufacturer, to begin executing whatever code is found there, or pointed to there. Sometimes this memory location contains code directly; e.g. upon coming out of reset, program execution begins at program address 0. Other times the fixed memory location is a vector, a location that holds the actual address of the beginning of the program; e.g. upon coming out of reset, the controller will load its program counter with the value found at program address 0xFFFE and thus start executing code at the address found in locations 0xFFFE and 0xFFFF (assuming a 16-bit program address, stored in 2 bytes). In the first instance you will have to make sure that your program has loaded at the specified start-up address, while in the second instance you will load your program wherever the program memory has been placed in the controller address space, and you will have to make sure that you then load that start-up address into the reset address vector. Note that the choice of start-up method is not up to you, but will be built in to the design of the µC you have chosen. AVR uses the first method, and Cortex M3 uses the second.

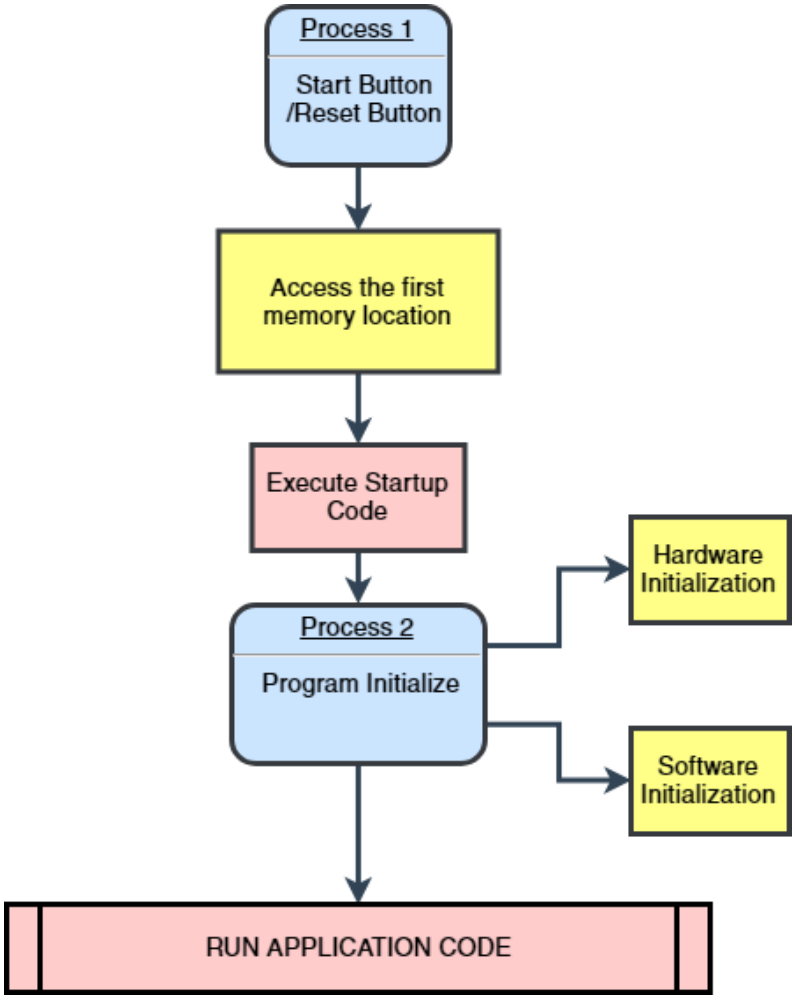


Figure 1: Start up Process flowchart

When an embedded program starts to run, there is usually a fair amount of initialization and housekeeping that must be done before the meat of the program begins. Most of this initialization is something that the average desktop programmer never sees, since it is handled by the computer boot code and operating system. But in an embedded system, it is just as likely as not that there is no operating system, and all boot code and other start-up code must be explicitly provided. Some very critical hardware may need to be initialized first e.g. hardware that controls memory access times and address maps, as well as system clock hardware. Then some software initialization may need to happen, such as setting up a stack pointer and perhaps copying data from non-volatile memory to volatile memory where it can be accessed and perhaps modified. After that will usually come another round of hardware initialization, setting up any peripheral devices that the system requires, and setting initial output states. Finally, yet another round of software initialization may occur.

This initialization is usually broken up into two sections, with the first hardware and software initialization steps often being done in what is known as the start-up code, and the later hardware and software steps being done in the user program. This delineation is more distinct in a C program, where the start-up code is invisible to the C program, being the code that happens before *int main()* is run, and ending in a jump or call to main() where the visible C program begins. After all the housekeeping is done then the system application is then executed. This is usually contained in the *int main()* section of the code

A NOTE ON THE EXAMPLE PROGRAMS
Lessons may include code sections with short example programs. The examples will start with the simplest concepts and add some concepts in each succeeding program.

Example of a typical "Hello, World" code.

```
#include <stdio.h> //Inclusion of a header file that contains the printf()function

int main()         // The main function that is run after initialization
{

   printf("Hello, World!"); // printf() displays the string inside quotation
   return 0;
}
-------------------------------------------------------------------------------------
Hello, World!
```

BASIC CPU ORGANIZATION

When you write a program for your microcontroller you are really writing a program that is executed by the µC CPU (central processing unit), executing various instructions designed into the CPU instruction set, so it's worth a bit of time to take a brief look at the innards of a typical µC CPU. If the program is written in ASM, the programmer writes the program using these CPU instructions directly. If the program is written in a high-level language, the compiler translates the HLL lines into corresponding sequences of CPU instructions. What follows is not in any sense meant to be a comprehensive discussion of CPU architecture design. Rather, it is just enough detail to make the sections that follow understandable. There's lots of additional detail online and in datasheets.

In the simplest sense, a CPU is that part of the microcontroller that executes instructions. It does this in a series of steps:

• Fetch an instruction from the "next instruction" memory location pointer

• Execute that instruction

• Advance the "next instruction" pointer accordingly

Every computer program is just a repetitive execution of this sequence.

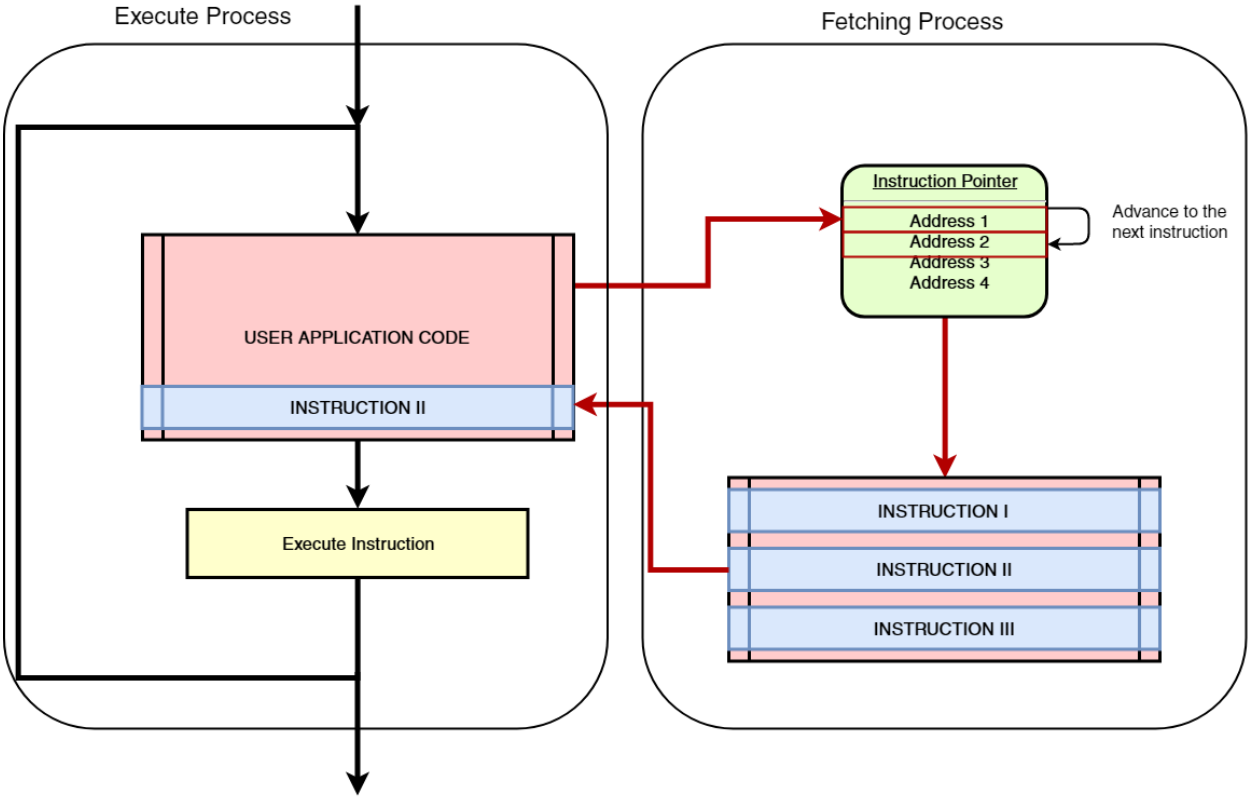


Figure 2: Fetch and execute cycle of a microcontroller

CPU REGISTERS

Any CPU will have a set of on board registers, which can be viewed as very fast memory locations. These registers will either be addressed automatically or they will be addressed by a few bits in the instruction operation code. A typical CPU will have the following types of registers:

1. **Data Registers**: These registers act as sources and destinations for arithmetic, logic and shifting instructions.
2. **Addressing Registers**: These registers are used to hold addresses for memory data accessing.
3. **General-purpose Registers:** These registers are specially designed and can be configured to be used as data or addressing registers.
4. **Stack Pointer:** This is an address register which points to a section of memory that is used by the CPU to keep track of subroutine and interrupt, calls and returns. It is also possible for the user program to access data stored on the stack pointer. Placing/storing data onto the stack is called *Pushing* and removing/accessing data from the stack is called *Popping*.
5. **Program Counter/Instruction Pointer:** This is the address register that points to the current (or next) instruction to be executed. It will automatically advance to point to the next instruction in a program, and will automatically be adjusted based on program jump, call and return instructions.

CPU INSTRUCTION TYPES

A microcontroller CPU will execute a range of instructions, some of which manipulate data, some of which affect program control or CPU behaviour, and some of which perform other actions. Here is a brief overview of the main types of CPU instructions, but it is important to note that CPU instructions vary with different µCs.

DATA MANIPULATION INSTRUCTIONS

Arithmetic Instructions

Arithmetic instructions allow the CPU to add and subtract numbers, and in many cases to multiply and divide numbers. These numbers may be constants, fixed for all time as the program runs, or they may be variable, modifiable values held in data memory or registers. Signed numbers are represented in 2s complement form.

• ADD: A=B+C

• SUBTRACT: A=B-C

• COMPARE: B < C, B>=C, B==C

• NEGATE: A= -B; this is usually done by taking the signed number subtracting it from zero i.e. A = 0-B.

• INC and DEC: A++ = (A = A+1), A-- = (A = A-1); increment and decrement instructions, which are shorthand instructions for adding or subtracting 1

Logic Instructions

These are usually done when dealing with digital inputs and output signals that the µC handles. Digital signals are generally represented using 1s and 0s, i.e. 1 = ON, and 0 = OFF. This is what is known as binary format. These instructions will typically work on data that is the natural word size of the CPU, or smaller, e.g. 8 bits on an AVR, and 32, 16 or 8 bits on an ARM Cortex M3.

• COMPLEMENT (NOT): sets every 1 bit in A to 0, and every 0 bit in A to 1

A = 00110110

X = ~A

X = 11001001

• AND (&): Produces the bitwise AND of A and B, i.e. 1 in every bit position where both A and B are 1, 0 elsewhere

A = 00110110

B = 01101100

X = A & B

X = 00100100

• OR (|): Produces the bitwise OR of A and B, i.e. 1 in every bit position where either A and B are 1, 0 elsewhere

A=00110110

B=01101100

X = A | B

X=01111110

• XOR (^): Produces the bitwise XOR of A and B, i.e. 0 in every bit position where A and B are the same, 1 where they are different

A=00110110

B=01101100

X = A ^ B

X=01011010

Shifting Instructions

• SHIFT LEFT/RIGHT, (<</ >>):

Shifts each bit one position to the right or left. For left shifts the "empty" bit position (D0, least significant bit, LSB) will be loaded with 0. For right shifts the empty bit position (most significant bit, MSB) will usually keep the value it had (sign extension) but there may be versions of the instruction that shift a 0 into the MSB. For both directions, the bit that is shifted out is usually shifted into the Carry bit, which provides the linkage for multi-stage shifts.

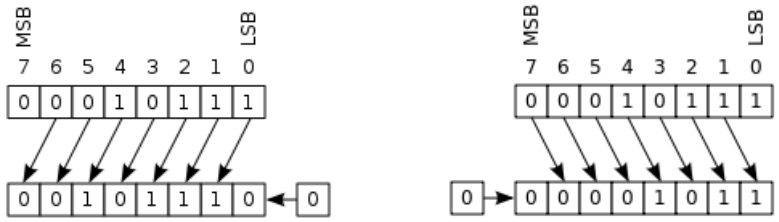


Figure 3: Bit shifting operations, a-left shift, b-right shift

A=00110110

X = A >> 3

X=00000110

A=00110110

X = A <<2

X=11011000

Example Codes:

```
#include<stdio.h>

int main()
{
        int a=12,b=10;

        printf("\nNumber1 complement: %d", ~a);
        printf("\nNumber1 + Number2 : %d",a + b);
        printf("\nNumber1 - Number2 : %d",a - b);
        printf("\nNumber1 AND Number2 : %d",a & b);
        printf("\nNumber1 OR  Number2 : %d",a | b);
        printf("\nNumber1 XOR Number2 : %d",a ^ b);

        return(0);
}
```

```
-----------------------------------------------------------------------------------------
a = 12
b = 10
---------------------------------
a in Binary : 0000 0000 0000 1100
b in Binary : 0000 0000 0000 1010
---------------------------------

~a        : 1111 1111 1111 0011 = -13
a + b     : 0000 0000 0001 0110 = 22
a - b     : 0000 0000 0000 0010 = 2
a | b     : 0000 0000 0000 1110 = 14
a & b     : 0000 0000 0000 1000 = 8
a ^ b     : 0000 0000 0000 0110 = 6
```

```
#include<stdio.h>

int main()
{
        int a = 60;

        printf("\nNumber is right Shifted By 1 Bit  : %d",a >> 1);
        printf("\nNumber is right Shifted By 2 Bits : %d",a >> 2);
        printf("\nNumber is left Shifted By 3 Bits : %d",a << 3);

        return(0);
}
```

```
-----------------------------------------------------------------------------------------
a = 60
---------------------------------
a in Binary : 0000 0000 0011 1100
---------------------------------
a >> 1    : 0000 0000 0001 1110   = 30
a >> 2    : 0000 0000 0000 1111   = 15
a << 3    : 0000 0001 1110 0000   = 480
```

What happens when you bit shift negative numbers? Repeat the last code above but use a negative number instead of 60.

How/ Where to use Bit-wise operations and techniques.

We have learnt different Bitwise Operations and in this section we are going to learn how to apply them when dealing with reading inputs and outputs from a microcontroller. A key technique that is used in this process is known as masking. It is the process or operation to set bit on to off or off to on in a byte, nibble or word. To mask means to block and through masking we are able to retain the data we require and block the rest.

```
      Num 1: 1001 1101 1001 0101 // Data in output register
OR (|) Mask: 0000 1000 0000 1000 // Masking data to set certain outputs to 1 and perform an OR
--------------------------------
     Result: 1001 1101 1001 1011 // If it was 0 -> 1 if it was 1 remains


      Num 1: 1000 0001 1110 1011 // Data in output register
AND(&) Mask: 0000 0000 0000 1111 // Masking data to set all pins to 0 other than the first 4 bits
--------------------------------
     Result: 0000 0000 0000 1011 // first 4 bits remain while the rest are reset to 0.
```

Numbering systems i.e. Binary, Decimal and Hexadecimal:

Overview of the different systems mainly, binary, decimal and hexadecimal.

Lesson 3:

So far we have dealt with bitwise operations and next we will now apply this knowledge in handling inputs and outputs using the General purpose input output (GPIO) Ports and Registers on the AVR ATmega32

Overview of the ATmega 32 GPIO:

AVR ATmega32 has 32 pins constituting four ports. The ports are listed below: PORT A, PORT B, PORT C and PORT D. Each port has 8 pins where each pin can be used as general purpose inputs/outputs. Additionally, if a pin is configured as an input, it can further be configured to enable a weak internal pullup resistor on the input. So the three choices are output, input, or input with internal pullup. These pins can be configured and used as input or output using the three I/O registers for each port. These registers are listed below:

1. DDRx registers: These are used to decide the direction of the pins, i.e. whether the pins will act as input pins or as output pins

2. PINx registers: These are used to read the logic level on the port pins.

3. PORTx registers: These are used to set the logic on the pins HIGH or LOW.

(Where x can be A, B, C or D depending on which port registers are being addressed), e.g. DDRB, PINB, PORTB – registers for port B,

NB: Each pin also has some special functionality associated with it as well. The pin diagram of ATmega32 is shown in the figure below. The four ports, their pins and the special functions associated with each pin can be seen in the figure.

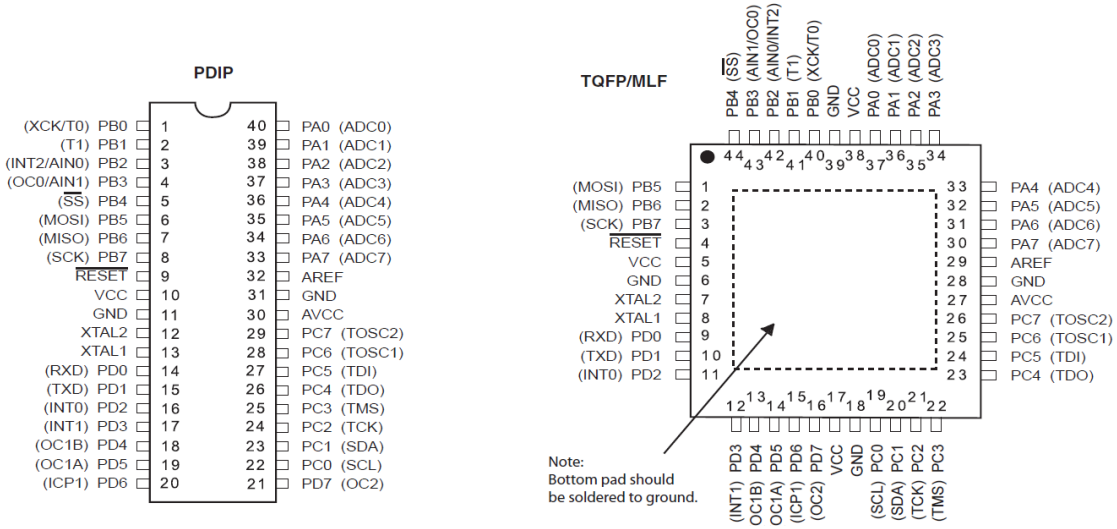


Figure 4: Pin Diagram of ATmega32

I/O Port Registers:

Next we will take a closer look at the 3 port registers to understand how to use them. As mentioned before there are three I/O registers associated with each port. These are as follows:

Data direction register DDRx, Input pins address register PINx and Data register PORTx; Where x can be A, B, C or D depending on which port is being addressed.

DDRx : Data Direction Registers:

These are 8 bit registers that are used to configure the pins of the ports as input or output. Writing a ONE in the bits sets those specific pins as OUTPUT pins whereas writing a ZERO to the bits in this register sets those specific pins as INPUT pins. All bits in these registers can be read as well as written to and the initial value of these bits is zero.

Example:

Setting Port D as an output port: All the pins in port D will be used as outputs.

```
DDRD = 0xFF; // 0xFF = 0b 1111 1111.
```

Writing 1 to the bits of this register configures the pins corresponding to those pins as output pins. Here we have configured all 8 pins of Port D as output pins by writing 1 to each of them.

Setting Port D as an input port:

```
DDRD = 0x00; // 0x00 = 0b 0000 0000
```

Writing 0 to the bits of this register configures the pins corresponding to those pins as output pins. Here we have configured all 8 pins of Port D as input pins by writing 0 to each of them.

PORTx : Data Registers

These are 8 bit registers that are used to put the pins of the ports in logic HIGH or logic LOW state. Writing a ONE to the bits in this register puts a HIGH logic (5V) on those pins whereas writing a ZERO to the bits in this register puts a LOW logic (0V) on those pins. All bits in these registers can be read as well as written to. Initial value of these bits is zero.

Example:

We will use PORTx register of Port D to write a value 0x55 to Port D.

```
PORTD = 0x55// 0x55 = 0b 0101 0101
```

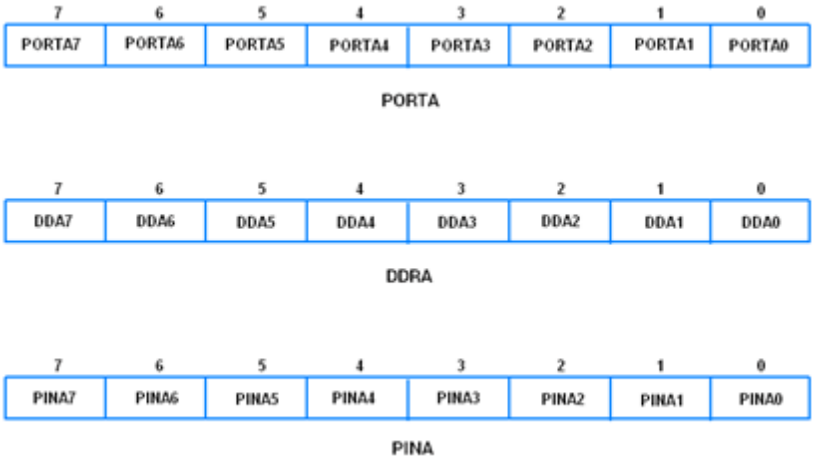
PINx : Input Pins Address Registers

These are 8 bit registers that are used to read the values on the specific pins of the port. These bits are read only bits and cannot be written to.

Example:

We will read the value on Port D in an 8 bit variable named 'port_value'.

```
port_value = PIND;
```



NB: it is important to note that what makes bit manipulation non-trivial is that bits do not usually exist alone, but exist within bytes (8 bits). Thus it becomes necessary, when writing individual bits to configure them as inputs or to output from an individual pin, you need to avoid changing other bits within the same byte/word. When reading individual bits you also need to avoid reading other bits within the same byte/word. The bit wise manipulation methods i.e. AND, OR etc., as discussed in an earlier section, are what allow us to do these things.

Digital Outputs:

Coding Example: EMBEDDED HELLO WORLD

In order to put the knowledge we have gained to practice we will implement the standard first program on an embedded platform which is blinking LED.

A program to blink an LED must do the following things:

• Set a GPIO (general purpose input/output) pin to be an output

• Drive this output pin (connected to an LED) alternately high (1) and low (0)

• Delay a human-discernable length of time between each output pin change

Circuit Diagram:

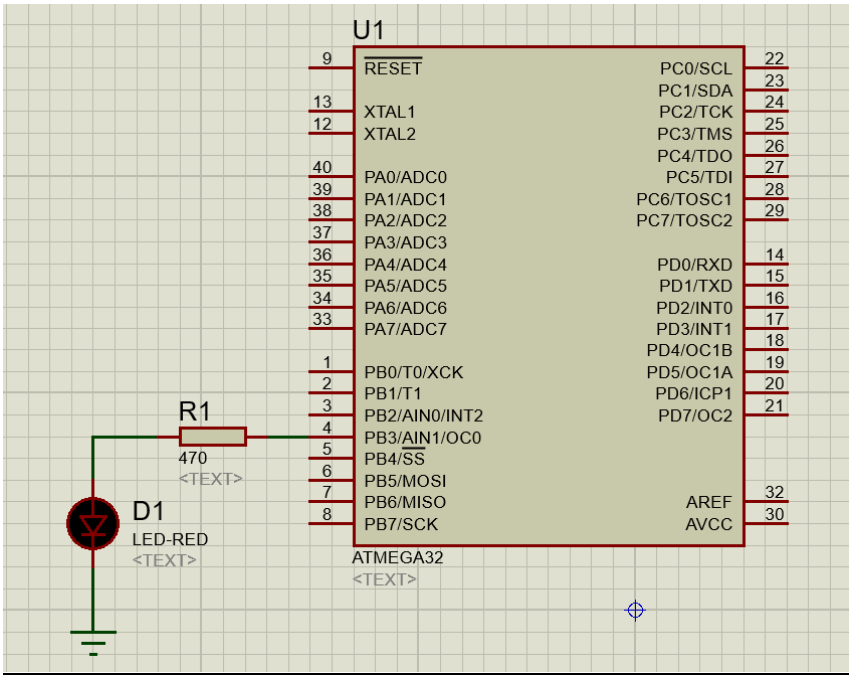


Figure 5: Simple LED connected to an Atmega32 in PROTEUS

Open Proteus and implement a circuit diagram as shown in the figure above.

Most of the LEDs require around 10mA current to glow properly. If current greater than maximum rated current of the LED flows through it, the LED will get permanently damaged. The maximum current rating for LEDs is usually around 20-30mA. Refer the datasheet of the LED you are using for this rating. In order to limit the current flowing through the LED, we connect a resistor in series with it. Assuming that the LED has a 0.7V forward voltage drop, and that 10mA current is sufficient for the LED to glow properly, the value of resistance required can be calculated as follows:

$$R = \frac{Voltage\ Supplied - Forward\ Voltage\ Drop}{Current\ Flow\ through\ LED} = \frac{5v - 0.7v}{0.01A} = 430\Omega$$

We generally choose a commercial resistor close in value to the ideal thus 470Ω is used.

<u>Code Section:</u>

Step 1 - Create Empty Program

Atmel Studio will create an empty program for you, including all the chip-specific data for the microcontroller you specify. If you are using a different compiler, or even a different language, the documentation for the compiler will tell you how to create an empty program.

Step 2 - Write the LED Blink Loop

A flowchart representation of a simple implementation of our code is as shown:

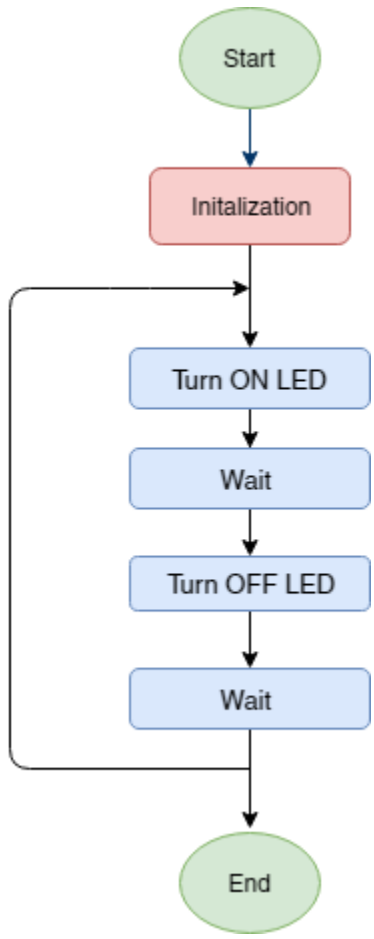


Figure 6: Blink LED flowchart

Programs can be written using various methods you can think of a more efficient way to achieve the above functionality using XOR.

As we proceed to write the code, we need to consult the data sheet of the Atmega32 so as to check how to properly address and deal with the pin assignments. Open the data sheet and proceed to the section containing the I/O instructions.

```c
/*Embedded Hello World*/

#define F_CPU 8000000UL // Defining the clock speed which we will use for the uC
#include <avr/io.h>     // including header file for input and output definitions
#include <util/delay.h> // including header file for the delay function

int main(void)
{
   /* Setting all pins in PORTB to OUTPUTs*/
   DDRB = 0xFF;   // we generally use binary or hexadecimal i.e. 0xFF = 0b11111111

    /*
    In order to make only PB3 as output while keeping the other pins unchanged.
    DDRB = DDRB | (1<<PB3); NB: PB3 = 3
    */
    while(1)
    {
        PORTB = PORTB | (1<<PB3);      /* Making PB3 high. This will make LED ON */
        _delay_ms(100);
        PORTB = PORTB & (~(1<<PB3));   /* Making PB3 low. This will make LED OFF */
        _delay_ms(100);
    }
          return 0;
}
```

```
PORTB = PORTB | (1<<3)

      PORTB: 0000 0000 // Initial state of PORTB
OR (|) Mask: 0000 1000 // (1<<PB3) left shifts 1 by 3 spaces resulting to 0000 00001 -> 0000 1000
--------------------------------
      PORTB: 0000 1000 // PB4 is set to 1(5V)

-------------------------------------------------------------------------------------------------

PORTB = PORTB & (~(1<<3))

       PORTB: 0000 1000 // Initial state of PORTB
AND (&) Mask: 1111 0111 // (~(1<<PB3)) converts 0000 1000 -> 1111 0111
--------------------------------
      Result: 0000 0000 // PB4 is set to 0(0V)
```

Step 3 – Compile, upload and simulate the code in Proteus

NB: In the implementation of various functionalities using the ATmega 32, there are predefined header files such as Stdio.h, avr/io.h, util/delay.h, and may others that are used and which we will refer to. The website below is the official access page for such library files and contains detailed information of the various functions in each header file. You can access and include any necessary files in your project from in this website unless stated otherwise.

http://www.nongnu.org/avr-libc/user-manual/modules.html

WHAT NEXT?

After completing the "Embedded Hello World", we can safely explore a system application such as: traffic lights.

2st Report: Implement a simple time based traffic light system
Max 5 page (Excluding title page and reference page)
2 Weeks submission Deadline: Submission Date
Include diagrams, pictures and flowcharts.

```c
/*Embedded Hello World - XOR Version*/

#define F_CPU 8000000UL // Defining the clock speed which we will use for the uC
#include <avr/io.h>     // including header file for input and output definitions
#include <util/delay.h> // including header file for the delay function

int main(void)
{
   DDRB = (1<<PB3);
   while(1)
   {
     PORTB ^= (1<<PB3); // Remember X ^= Y means X = X^Y or X += Y means X = X+Y
     _delay_ms(100);
   }
}
```

```
PORTB ^= (1<<3)

      PORTB: 0000 0000 // Initial state of PORTB
XOR(^) Mask: 0000 1000 // (1<<PB3) left shifts 1 by 3 spaces resulting to 0000 00001 -> 0000 1000
--------------------------------
      PORTB: 0000 1000 // XOR set 0 if the bits are the same and 1 if they are different

-------------------------------------------------------------------------------------------

      PORTB: 0000 1000 // the second time around PORTB is 0000 1000
XOR(^) Mask: 0000 1000 // (1<<PB3) left shifts 1 by 3 spaces resulting to 0000 00001 -> 0000 1000
--------------------------------
      PORTB: 0000 0000 // XOR set 0 if the bits are the same and 1 if they are different
```

Using XOR results in a shorter code but both the ON and OFF cycles must have the same delay time as opposed to the first method where we could directly control both time delays. It is important to note: It is usually best to have short more efficient code. At first you can write your code in a way that is long but easier to understand. As you advance and your knowledge expands, try and head towards shorter and more efficient ways of writing code.

Lesson 4

Program Decision Statements; If –else, If-else if else and Switch-case

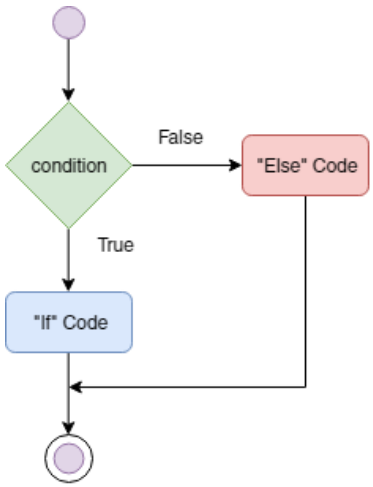
In introduction to programming, a key element that helps introduce a lot of control in you program are Decision statements. These are ways in which we enable the program to make decisions, depending on certain conditions.

If – else Statements: An **if** statement can be followed by an optional **else** statement, which executes when the Boolean expression is false.

Syntax:

```
if(boolean_expression)
 {// statement(s) will execute if the boolean expression is true}
else
 {// statement(s) will execute if the boolean expression is false}
```

Flowchart:



Example Code:

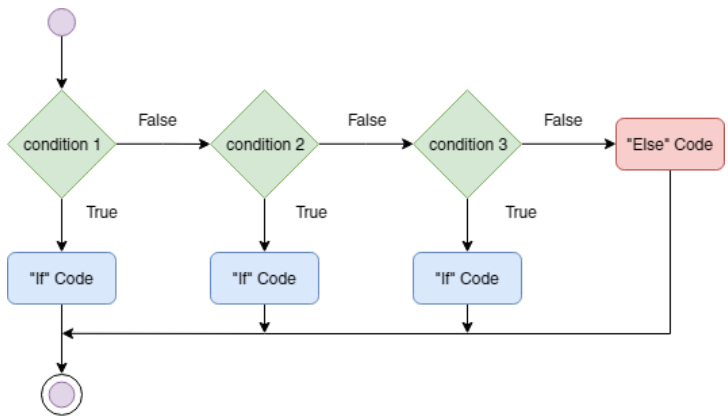
```
include <stdio.h>

int main ()
{
   int a = 100;

   if(a < 20) // check the boolean condition
     {// if condition is true then print the following
      printf("a is less than 20;\n");
     }
   else
     { // if condition is false then print the following
      printf("a is not less than 20;\n");
     }
   printf("value of a is %d", a);
   return 0;
}
```

<u>If-else-if-else statements:</u> An **if** statement can be followed by an optional **else if...else** statement, which is very useful to test various conditions using single if...else if statement.

Syntax:

```
if(boolean_expression)
 {// statement(s) will execute if the boolean expression is true}
else if(boolean_expression)
 {// statement(s) will execute if the boolean expression is true}
...
else
 {// statement(s) will execute if all the boolean expression are false}
```

Flowchart:



Example Code:

```
include <stdio.h>
int main ()
{
   int a = 100;// local variable declaration

   if(a == 10) // check the boolean condition
    {// if condition is true then print the following
      printf("Value of a is 10\n");
    }
   else if(a == 20)
    {// if else if condition is true
      printf("Value of a is 20\n");
    }
   else if(a == 30)
    {// if else if condition is true
      printf("Value of a is 30\n");
    }
   else
    {// if none of the conditions is true
      printf("Value is not matching \n");
    }
   printf("The exact value is: %d",a);
   return 0;
}
```

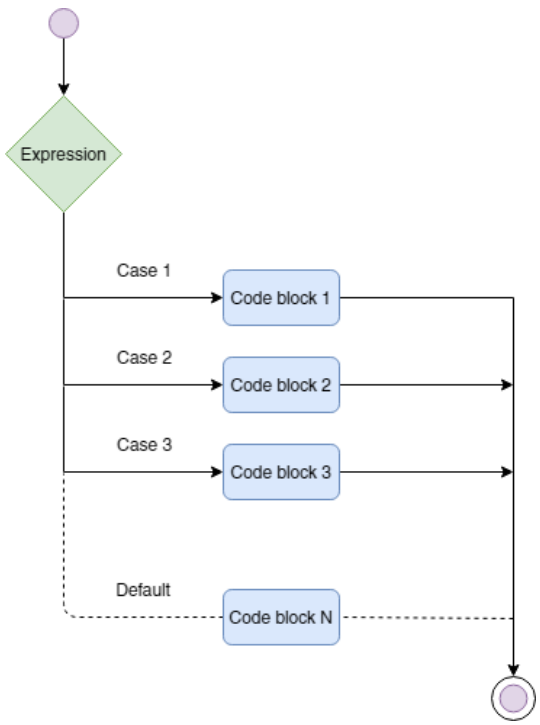
Switch case:

Switch case statement allows a variable to be tested for equality against a list of values. Each value is called a case, and the variable being switched on is checked for each switch case.

Syntax:

```
switch(expression) {

   case constant-expression  :
      statement(s);
      break; /* optional */

   case constant-expression  :
      statement(s);
      break; /* optional */

   /* you can have any number of case statements */
   default : /* Optional */
   statement(s);
}
```

Flowchart:



Example Code:

```
#include <stdio.h>

int main () {

   /* local variable definition */
   char grade = 'B';

   switch(grade) {
      case 'A' :
```

```
        printf("Excellent!\n" );
        break;
     case 'B' :
        printf("Well done\n" );
        break;
     case 'C' :
        printf("You passed\n" );
        break;
     case 'D' :
        printf("Better try again\n" );
        break;
     default :
        printf("Invalid grade\n" );
   }

   printf("Your grade is  %c\n", grade );

   return 0;
}
```

Digital Inputs:

Coding Example: Button Press LED

The next step is to explore the implementation of these two decision statements; if-else and switch case in an embedded system. We will do this by checking to observe whether an input has been received by the Atmega32 via a button press and deciding to light an LED based on this input.

The program must do the following things:

• Initializes and set the relevant GPIO (general purpose input/output) pins to be input and output

• Check whether the input pin/button has been pressed.

• Depending whether the button has been pressed, set the output pin HIGH/LOW

Circuit Diagram:

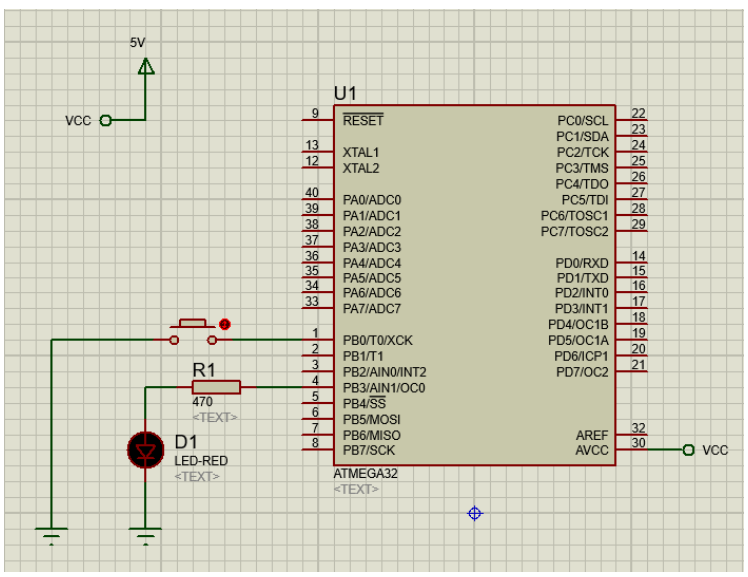


Figure 7: Simple Push button and LED connection

Open Proteus and implement a circuit diagram as shown in the figure above.

Code Section:

Step 1 - Create Empty Program

Atmel Studio will create an empty program for you, including all the chip-specific data for the microcontroller you specify. If you are using a different compiler, or even a different language, the documentation for the compiler will tell you how to create an empty program.

Step 2 - Write the Push button LED Loop

A flowchart representation of a simple implementation of our code is as shown:

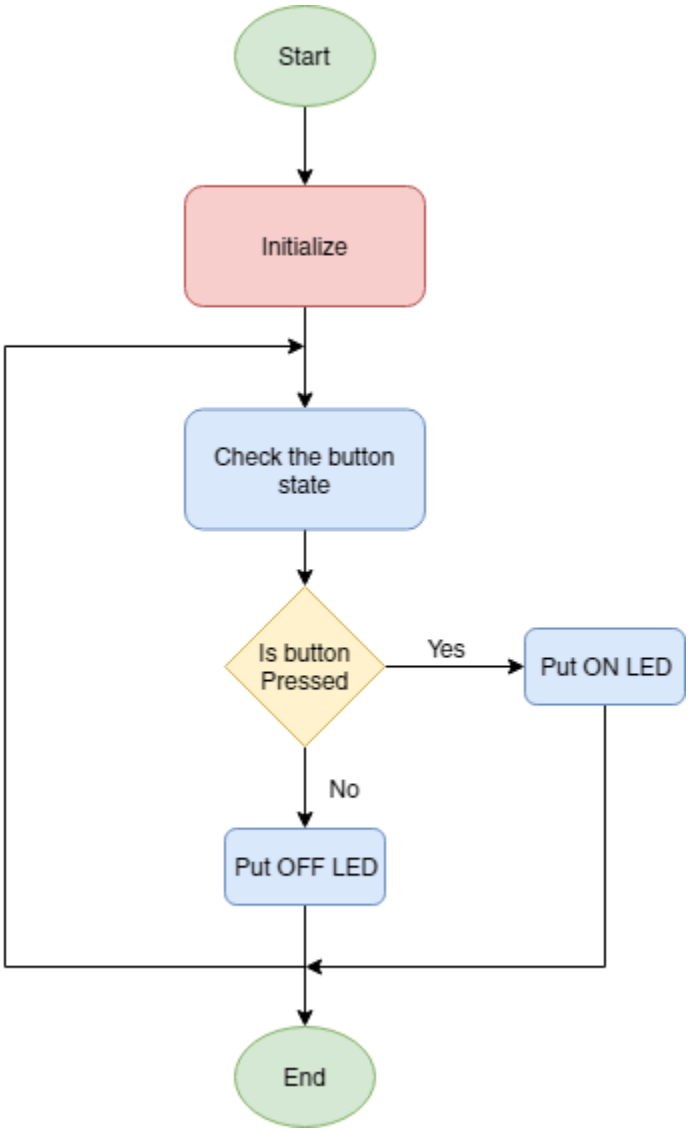


Figure 8: Blink LED flowchart

It is always important to consult the data sheet for the microcontroller to check how to properly obtain input data and to active pullup resistors. Open the data sheet and go to the section containing the I/O instructions.

```c
/* Push_button.c
 * Input Pin Programming */

#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>

int main(void)
{
    DDRB = DDRB | (1<<PB3);      /* Make PB3 as output pin */
    DDRB = DDRB & (~(1<<PB2));   /* Make PB2 as input pin */
    PORTB = PORTB | (1<<PB2);    /* Enable pull-up on PB2 by writing 1 to it */
    int pin_status;
    while(1)
    {
     pin_status = ~PINB & (1<<PB2); /*Read status of pin PB2 */
     if(pin_status)                 /* Transmit status of pin PB2 on to pin PB3 to drive LED.*/
      {
         PORTB = PORTB | (1<<PB3);  /*Switch is open, pin_status = 1, LED is ON */
      }
    else
      {
         PORTB = PORTB & (~(1<<PB3));/*Switch is closed, pin_status = 0, LED is OFF */
      }
    }
    return 0;
}
```

```
DDRB = DDRB | (1<<3)

       DDRB: 0000 0000 // Initial state of DDRB
OR (|) Mask: 0000 1000 // (1<<PB3) left shifts 1 by 3 spaces resulting to 0000 00001 -> 0000 1000
--------------------------------
      DDRB: 0000 1000 // PB4 is set to 1(5V)

-------------------------------------------------------------------------------------------

DDRB = DDRB & (~(1<<2))

        DDRB: 0000 1000 // Initial state of PORTB
AND (&) Mask: 1111 1011 // (~(1<<PB2)) converts 0000 0100 -> 1111 1011
--------------------------------
      Result: 0000 1000 // PB4 is set to 0(0V)
```

It should be noted that the setting PB2 as an input might be unnecessary since PB2 is by default 0 i.e. an input but it is always best to distinctly set the pin in the code to avoid floating bits and to be sure of the status of the pin.

```
-------------------------------------------------------------------------------------------

Pin_status = ~PINB & (1<<2)

       ~PINB: 0000 0100 // if PINB is LOW -> 1111 1011, after ~PINB -> 0000 0100
AND (&) Mask: 0000 0100 // (1<<PB2) 0000 0100
--------------------------------
  Pin_status: 0000 0100 // The state of PB2 is recorded in the variable pin_status
```

Step 3 – Compile, upload and simulate the code in Proteus

WHAT NEXT?

After completing successful implementation of push button code we can now safely explore a system application such as: button combination lock.

Debouncing:

Lesson 5:

Program Loop Statements: for loop, while loop, do-while loop

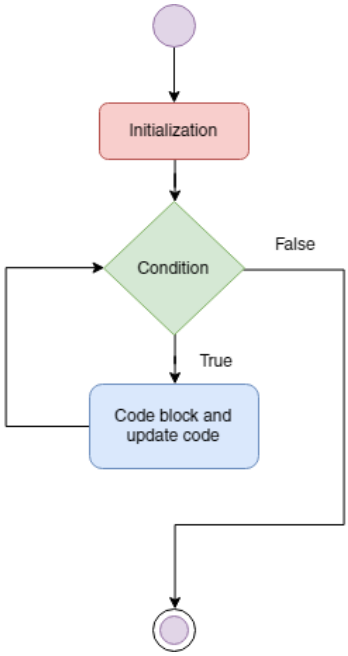
There are often situations, when a block of code needs to be executed several number of times. In general, statements are executed sequentially: but a loop statement allows us to execute a statement or group of statements multiple times. Given below is the general form of a loop statement.

For-loop: The initialization statement is executed only once. Then, the test expression is evaluated. If the test expression is false (0), for loop is terminated. But if the test expression is true (nonzero), codes inside the body of for loop is executed and the update expression is updated. This process repeats until the test expression is false. The for loop is commonly used when the number of iterations is known.

Syntax:

```
for(initialize; condition; update)
 {// statement(s) will execute if the condition expression is true}
```

Flowchart:



Code Example:

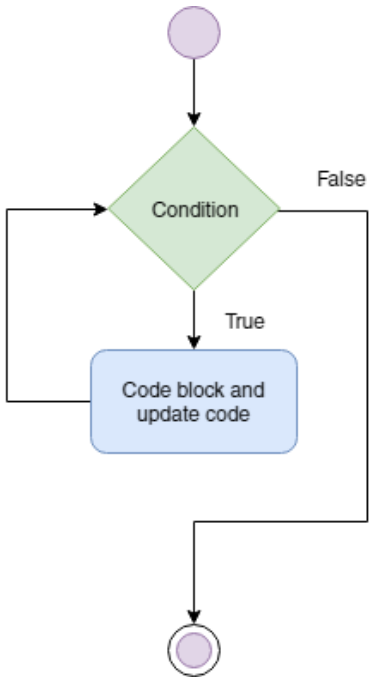
```
#include <stdio.h>

int main ()
{
 int limit = 10;

   for(int counter; counter < limit ; counter++ )
   {
      printf("This loop is on loop number %d.\n", counter);
   }

   return 0;
}
```

While-loop: The while loop evaluates the test expression. If the test expression is true (nonzero), codes inside the body of while loop is executed. The test expression is evaluated again. The process goes on until the test expression is false. When the test expression is false, the while loop is terminated.

Syntax:

```
while(condition)
 {// statement(s) will execute if the condition expression is true}
```

Flowchart:



Code Example:

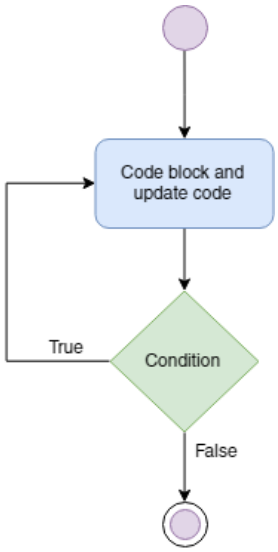
```
// Program to find factorial of a number
// For a positive integer n, factorial = 1*2*3...n

#include <stdio.h>
int main()
{
  int number = 10;
  long long factorial;
  factorial = 1;

  // loop terminates when number is less than or equal to 0
  while (number > 0)
  {
    factorial *= number;  // factorial = factorial*number;
    --number; // Update statement
  }
  printf("Factorial= %lld", factorial);

  return 0;
}
```

Do-While-loop: Unlike for and while loops, which test the loop condition at the top of the loop, the do...while loop in C programming checks its condition at the bottom of the loop. A do...while loop is similar to a while loop, except the fact that it is guaranteed to execute at least one time.

Syntax:

```
do
{code block}
while (condition);
```

Flowchart:



Code Example:

```
// Program to find factorial of a number
// For a positive integer n, factorial = 1*2*3...n

#include <stdio.h>

int main () {
  int a = 10;  /* local variable definition */
  int factorial = 1;  /* local variable definition */

  do{
   printf("value of factorial: %d\n", factorial);
   factorial = factorial * number;
   --a;

  }while (a > 0);
 return 0;
}
```

WHAT NEXT?

After completing successful implementation of factorial try and implement a power series and Fibonacci sequence generator

---

EMT 3104: Design Assignment I: 20mks

Task:

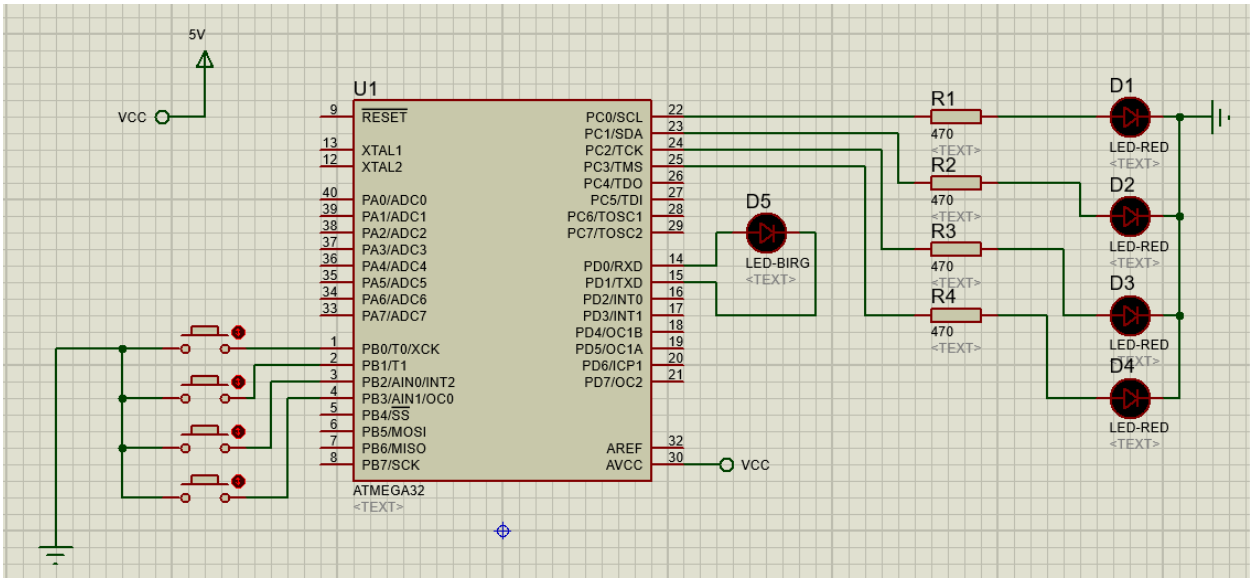
You are to write an embedded c program that will operate as an access system. The system lights a bi-color LED (BIRG) Green or Red depending on how the input buttons are pressed. If the correct passcode is pressed, the BIRG blinks green 4 times with a 500ms interval between each blink i.e. 500ms on, 500ms off. If the pass code is incorrect, the BIRG blinks red 4 times with the same 500ms sequence as mentioned above.

The password used needs to be 4 button presses, e.g. { 4, 3, 2, 1} means pressing the 4 buttons in that order. Any other order is considered incorrect and will make the BIRG flash red.

Each button press should progressively light the corresponding LEDs in PORTC. Once 4 buttons have been pressed, the system should check if the input sequence was correct and flash the BIRG correspondingly.

The system circuit is as shown below, it consists of 4 push buttons connected to PORTB (PB0 to PB3), 4 red LEDs connected to PORTC (PC0 to PC3) and a bi-color (Green/Red) LED connected to PORTD (PD0 and PD1). The bi-color LED lights Green if PD0 is HIGH and PD1 is LOW and it lights RED if PD0 is LOW and PD1 is HIGH.

Circuit:



Assessment:

As stated the assignment is worth 20 marks and the main criteria of marking include:

1. How well the code accomplishes the required task (8 mks)
2. Compactness, efficiency and neatness of the code (4 mks)
3. Design report (8 mks)

Submission:

You will have two submission items:

1. The AVR code
2. A design report in pdf format: Max 5 page (Excluding title page and reference page). Include important snippets of your code and a flowchart(s) where necessary.

. Save your work in a folder with your registration number e.g. E022-01-0111/2014_DESIGN I

. 2 Weeks submission Deadline: Submission Date. Submissions will be done in class.

NB:

- **Any plagiarism or copying of code or report will automatically nullify the affected students; share your original code at your own risk.**
- **Please report any potential issues or problems (e.g. difficulties in submission, issues requiring assistance etc) to the unit instructors before the submission deadline**
- **Any late submissions will not be accepted and the student will not receive marks on the assignment.**

Lesson 6

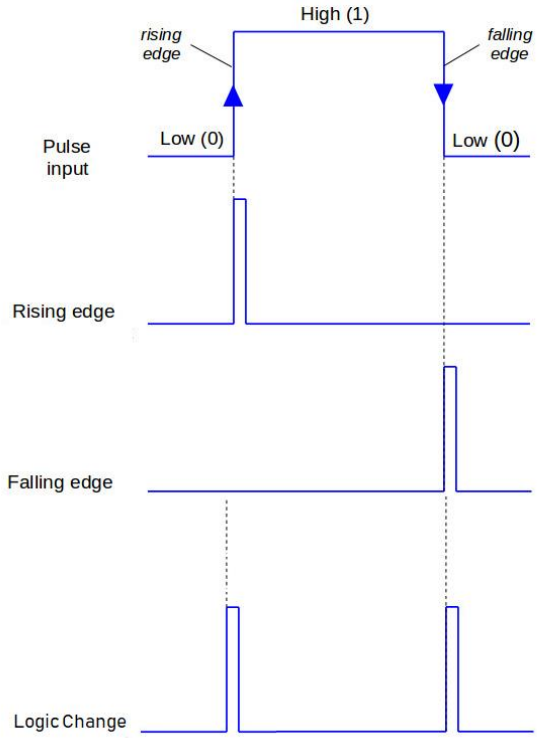
EMT 3104: Design Assignment Review

Interrupts: What are they and why are they useful?

As we have implemented various functionalities using the Atmega32 we observe that there are situations where we use delays to wait for certain actions to occur e.g. waiting to register an input from a button. This kind of situations forces the program to idle at a particular code section and in so doing delay other parts of the code. This inefficiency if further compounded if we have many input devices, the accumulated delays can lead to very slow running of the system. Similarly in more advanced applications with several processes running at the same time e.g. temperature regulation: reading user input, displaying data on an LCD, reading the temperature sensor/ADC output, and controlling the fan/heater output. The controller's bandwidth is to be divided among all these tasks in such a way that, to an end user, the functions seem to be executed in parallel. To achieve this kind of operation, we make use of interrupts.

As the name suggests interrupts, disrupts the CPU from the normal flow of instructions so that the CPU can carry out some specific functionality which is defined by the Interrupt Service Routine (ISR). After the ISR is complete, the CPU returns to its normal flow of instructions. This is very useful as it frees up the CPU to continue with other operations instead of inefficiencies in the code e.g. Input delays

There are 2 kinds of interrupts, i.e. External and Internal. **External interrupts** are triggered by hardware devices while **Internal interrupts** are triggered by sections of the code at specific intervals or other internal devices such as **Timers, Analog to Digital Converter(ADC), Communication functions(SPI and UART) etc.** For external interrupts the triggering of interrupt can occur in a variety of ways; Rising edge triggered, Falling edge triggered or Logic level triggered i.e. both on the rising and falling edges.



External Hardware Interrupts in AVR ATmega32

AVR ATmega32 has three external hardware interrupts on pins PD2, PD3 and PB2 which are referred as INT0, INT1 and INT2 respectively. Upon activation of these interrupts, the ATmega controller gets interrupted in whatever task it is doing and jumps to perform the interrupt service routine. External interrupts can be level triggered or edge triggered. We can program this triggering. INT0 and INT1 can be level triggered and edge triggered whereas INT2 can be only edge triggered.

Programming External Interrupt

We can enable / disable external interrupts by **GICR register.**

7	6	5	4	3	2	1	0
INT1	INT0	INT2	–	–	–	IVSEL	IVCE

- **Bit 7 – INT1 : External Interrupt Request 1 Enable:** 0 =Disable external interrupt, 1 =Enable external interrupt
- **Bit 6 – INT0 : External Interrupt Request 0 Enable:** 0 =Disable external interrupt, 1 = Enable external interrupt
- **Bit 5 – INT2 : External Interrupt Request 2 Enable:** 0 =Disable external interrupt, 1 =Enable external interrupt

MCU Control Register (MCUCR)

To define level trigger or edge trigger on external INT0 and INT1 pins MCUCR register is used.

7	6	5	4	3	2	1	0
SM2	SE	SM1	SM0	ISC11	ISC10	ISC01	ISC00

ISC01, ISC00 (Interrupt Sense Control bits)

These bits define the level or edge that triggers the INT0 pin.

ISC01	ISC00	Description
0	0	**The low level** on INT0 pin generates an interrupt request.
0	1	**Any logical change** on INT0 pin generates an interrupt request.
1	0	**The falling edge** on INT0 pin generates an interrupt request.
1	1	**The rising edge** on INT0 pin generates an interrupt request.

ISC11, ISC10 (Interrupt Sense Control bits)

These bits define the level or edge that triggers the INT1 pin.

ISC11	ISC10	Description
0	0	**The low level** on INT1 pin generates an interrupt request.
0	1	**Any logical change** on INT1 pin generates an interrupt request.
1	0	**The falling edge** on INT1 pin generates an interrupt request.
1	1	**The rising edge** on INT1 pin generates an interrupt request.

MCU Control and Status Register (MCUCSR): To define the INT2 interrupt activity, bit 6 of MCUCSR is used.

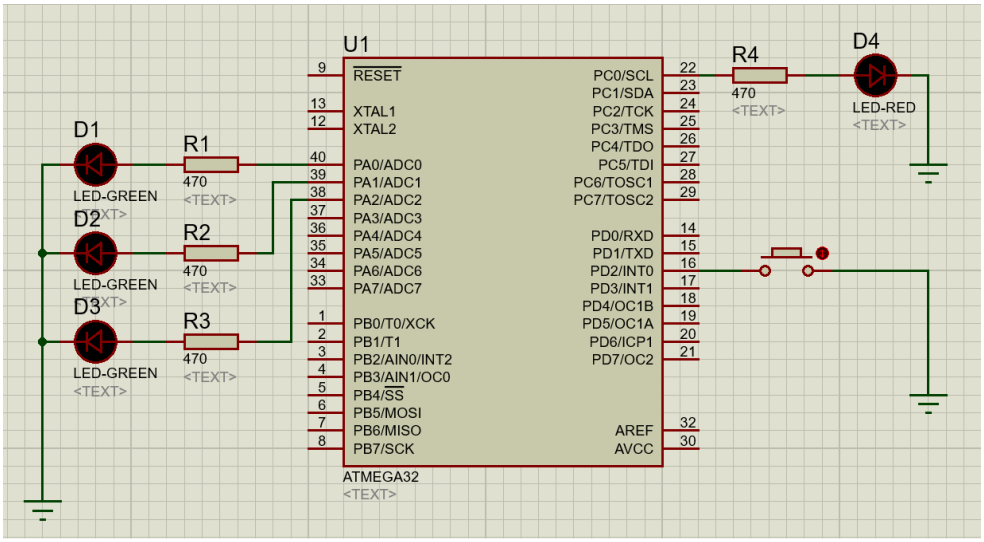
7	6	5	4	3	2	1	0
JTD	ISC2	–	JTRF	WDRF	BORF	EXTRF	PORF

ISC2 bit defines the INT2 interrupt triggering.

ISC2	Description
0	The falling edge on INT2 pin generates an interrupt request.
1	The rising edge on INT2 pin generates an interrupt request.

Example Implementation:
We will implement an interrupt based button that will operate a Red-LED, while a flashing loop is running. The circuit diagram for the exercise is as shown below

Circuit Section:



Code Section:

```c
/*
 * interrupt demo.c
 * Author : Michael Mureithi
 */
#define F_CPU 8000000UL
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>

/*Interrupt Service Routine for INT0*/
ISR(INT0_vect){
          PORTC ^= (1<<PC0); /* Toggle PORTC PC0 */
           delay ms(50);     /* Software debouncing control delay */
}

int main(void){
          /* Initialization */
          DDRA = 0x07;       /* Make PORTA as output PORT i.e. PA0, PA1, PA2*/
          DDRC = 1<<PC0;     /* Make PORTC as output PORT i.e. PC0*/
          DDRD = 0;          /* PORTD as input */
          PORTD = 1<<PD2;    /* Activate pull up resistor high */

          /* Interrupt setup */
          GICR = 1<<INT0;              /* Enable INT0*/
          MCUCR = 1<<ISC01 | 1<<ISC00; /* Trigger INT0 on rising edge */
          sei();                       /* Enable Global Interrupt */

          while(1){
                    /* flashing LED code*/
                    PORTA = 1<<PA0;
                    _delay_ms(200);
                    PORTA = 1<<PA1;
                    _delay_ms(200);
                    PORTA = 1<<PA2;
                    _delay_ms(200);
          }
}
```

WHAT NEXT?

Now that you understand the use of external interrupts, attempt the **EMT 3104: Design Assignment I,** with 3 input buttons connected to interrupt pins INT0, INT1, INT2 and with 3 output LEDs. Utilize external interrupts to read inputs from the buttons instead of wait delays.